

# Módulo 01 - Fundação Backend e Catálogo

---

## Objetivo

Entender como o projeto saiu de uma estrutura Django básica para uma API REST inicial de catálogo, com configuração de ambiente, CORS, serviços, permissões e testes.

## Commits Estudados

- `7b184a4` - estrutura monorepo e modelo base de brinquedos.
- `2002579` - configuração e CORS.
- `a60e2bb` - correção de sintaxe do modelo `Brinquedo` e migration.
- `f77b1cd` - API REST para catálogo de brinquedos.
- `8b82a77` - camada de serviço e permissões no `BrinquedoViewSet`.
- `51f8ebf` - configuração de ambiente e documentação de segurança.
- `67aac45` - testes de autenticação e validação da API de brinquedos.

## Aula 1 - O Primeiro Domínio Visível

No commit `7b184a4`, o projeto nasce como monorepo com `backend/`, app `catalogo` e modelo `Brinquedo`.

Código para ler:

- `backend/catalogo/models.py`
- `backend/catalogo/admin.py`
- `backend/setup/settings.py`
- `backend/setup/urls.py`

O ponto importante não é o tamanho do modelo. É a escolha de começar pelo agregado mais público do produto: o brinquedo alugável.

No estado atual, `Brinquedo` evoluiu bastante, mas a ideia central continua no mesmo arquivo:

```
class Brinquedo(models.Model):
    nome = models.CharField(max_length=200)
```

```
descricao = models.TextField()
preco_aluguel = models.DecimalField(max_digits=6, decimal_places=2)
ativo = models.BooleanField(default=True)
```

Conceito de engenharia:

- O catálogo é a porta de entrada do produto.
- Preço é `DecimalField`, não `FloatField`.
- A modelagem começa simples, mas já cria uma entidade que depois receberá estoque físico, imagens, categorias e kits.

## Aula 2 - Configuração Antes de Funcionalidade

O commit `2002579` adiciona CORS e prepara o backend para conversar com o frontend.

Código para ler:

- `backend/setup/settings.py`
- `backend/requirements.txt`
- `backend/catalogo/admin.py`

Ponto real do projeto:

- `corsheaders` entra em `INSTALLED_APPS`.
- `CorsMiddleware` entra no pipeline de middlewares.
- O admin começa a expor melhor o `Brinquedo`.

Essa decisão aparece depois como base para o cliente HTTP do frontend, que usa `credentials: "include"` no commit `b9a9a92`.

## Aula 3 - Migration Como Contrato de Banco

O commit `a60e2bb` corrige a sintaxe de `Brinquedo` e gera `backend/catalogo/migrations/0001_initial.py`.

Código para ler:

- `backend/catalogo/migrations/0001_initial.py`
- `backend/catalogo/models.py`

O aprendizado profissional aqui é simples: em Django, a modelagem vira contrato persistido por migration. Quando o modelo muda, a migration registra a intenção para o banco.

Correção real destacada:

- Este é um commit de fix, não de feature.
- Ele mostra a prática de corrigir o modelo antes de construir endpoints sobre uma base instável.

## Aula 4 - Primeira API REST com DRF

O commit `f77b1cd` cria a API REST de brinquedos.

Código para ler:

- `backend/catalogo/serializers.py`
- `backend/catalogo/views.py`
- `backend/catalogo/urls.py`
- `backend/setup/urls.py`

O caminho didático:

- `ModelSerializer` traduz `Brinquedo` para JSON.
- `ModelViewSet` entrega listagem, detalhe, criação, edição e remoção.
- `DefaultRouter` cria rotas REST de forma padronizada.
- `setup/urls.py` inclui as rotas em `/api/`.

Exemplo de raciocínio no código:

```
router.register(r"brinquedos", BrinquedoViewSet, basename="brinquedos")
```

Essa linha é pequena, mas define a superfície pública inicial do backend.

## Aula 5 - Serviço e Permissões

O commit `8b82a77` introduz `BrinquedoService` e permissões dinâmicas no `BrinquedoViewSet`.

Código para ler:

- `backend/catalogo/services.py`
- `backend/catalogo/views.py`

O projeto passa a separar responsabilidades:

- ViewSet decide HTTP, serializer e permissões.
- Service concentra acesso a dados e regra de consulta.
- Usuário anônimo pode listar e ver detalhes.
- Escrita fica restrita a admin.

No estado atual, a ideia aparece assim:

```
def get_permissions(self):
    if self.action in ["list", "retrieve", "disponibilidade"]:
        permission_classes = [AllowAny]
    else:
        permission_classes = [IsAdminUser]
    return [permission() for permission in permission_classes]
```

Decisão real de produto:

- Catálogo público precisa ser navegável sem login.
- Operação de catálogo precisa ficar protegida.

## Aula 6 - Ambiente, Segurança e Regras do Projeto

O commit `51f8ebf` cria ou atualiza:

- `AGENTS.md`
- `docs/PROJECT_CONTEXT.md`
- `docs/CODEX_RULES.md`
- `docs/SECURITY_CHECKLIST.md`
- `backend/.env.example`
- `backend/setup/settings.py`

Esse commit é importante porque transforma decisões em restrições de engenharia.

Regras que depois aparecem no código:

- autenticação oficial por sessão Django;
- validação crítica no backend;
- cliente só acessa os próprios dados;
- admin tem rotas próprias;

- estoque por unidade física;
- preço, frete e disponibilidade calculados no backend.

Conexão com commits futuros:

- `a0e6c5b` implementa sessão.
- `eeb170a` remove JWT.
- `aa5286e` reserva unidade física.
- `13464c8` implementa contrato e aceite.
- `2c50443` calcula entrega no backend.

## Aula 7 - Testes Como Fechamento de Comportamento

O commit `67aac45` aumenta `backend/catalogo/tests.py`.

Código para ler:

- `backend/catalogo/tests.py`

O foco dos testes:

- anônimo lista brinquedos;
- anônimo não cria brinquedo;
- usuário comum autenticado não cria brinquedo;
- admin cria brinquedo;
- API valida campos obrigatórios;
- campos somente leitura não são aceitos como fonte de verdade.

Ponto de engenharia:

- A regra "catálogo público, escrita admin" deixa de ser só intenção.
- O teste vira proteção contra regressão.

## Revisão do Módulo

Você deve conseguir responder:

- Onde a API pública de brinquedos é registrada?
- Por que `BrinquedoService` existe?
- Qual diferença entre permissão HTTP e validação de serializer?

- Que commit prova a primeira disciplina de testes?
- Qual risco existe em liberar escrita de catálogo para usuário comum?

## Exercício Prático

Abra `git show 8b82a77` e identifique:

- a criação de `BrinquedoService` ;
- a mudança em `BrinquedoViewSet` ;
- a diferença entre listar catálogo e alterar catálogo.

Depois abra `git show 67aac45` e conecte cada teste a uma regra da view.